

Clase 1

Tema 2

Ejercicio 1:

Código	Complejidad	Justificación
Fragmento (a)	$O(0)$	No depende de n , siempre ejecuta el mismo resultado
Fragmento (b)	$O(n^2)$	Un bucle anidado dentro de otro, ambos recorren la entrada
Fragmento (c)	$O(\log n)$	Al n dividirse por 2, el problema se reduce a una fracción (la mitad) en cada iteración
Fragmento (d)	$O(2^n)$	Al agregar otro bucle for, el trabajo se duplica una vez más y crece más rápido que el caso del fragmento (b)

Ejercicio 2:

Comparar par de elementos: El orden de complejidad sería de $O(n^2)$ ya que al haber dos bucles anidados la carga de trabajo crece de forma exponencial.

```
Para i := 0 Hasta N-2 Hacer
    Para j ← i+1 Hasta N-1 Hacer
        Si A[i] == A[j] Entonces
            duplicado := VERDADERO
        FinSi
    FinPara
FinPara
Si duplicado == VERDADERO Entonces
    Escribir "El arreglo tiene elementos duplicados"
Sino
    Escribir "El arreglo no tiene elementos duplicados"
FinSi
```

Comparar elementos ordenados: En este caso, al hacer un bucle de ordenamiento y luego uno de búsqueda, tenemos dos tipos de complejidad, para el primero es $O(n^2)$ y para el segundo $O(n)$, por lo que al final el orden de complejidad terminaría siendo $O(n^2)$ ya que $O(n)$ termina siendo despreciable en el resultado final.

```
Para i := 0 Hasta N-2 Hacer
    Para j := i+1 Hasta N-1 Hacer
        Si A[i] > A[j] Entonces
            aux := A[i]
            A[i] := A[j]
            A[j] := aux
        FinSi
    FinPara
FinPara
duplicado := FALSO
Para i := 0 Hasta N-2 Hacer
    Si A[i] = A[i+1] Entonces
        duplicado := VERDADERO
    FinSi
FinPara
Si duplicado = VERDADERO Entonces
    Escribir "El arreglo tiene elementos duplicados"
Sino
    Escribir "El arreglo no tiene elementos duplicados"
FinSi
```

Caso $n = 1.000.000$: Como los dos algoritmos poseen el mismo orden de complejidad ($O(n^2)$) el resultado final no varía en nada ya que poseen el mismo orden. Pero si tuviéramos en cuenta tiempos de ejecución por ejemplo, el más conveniente sería el primero ya que no necesita ordenar el arreglo y simplemente haría la búsqueda de elementos duplicados, a diferencia del segundo que primero necesita ordenar todo el arreglo para luego hacer la búsqueda; y como los dos hacen el recorrido hasta el final del arreglo vuelve más conveniente a la primer solución. Pero para el caso de que necesitáramos el arreglo ordenado para otras implementaciones, el segundo caso sería más conveniente.